PT. LANGGENG SEJAHTERA KREASI KOMPUTASI

Standar Arsitektur Sistem dan Penyusunan Dokumen Teknis

Menyatukan empat Development Standard Stack menjadi satu arsitektur sistem, dan membakukan cara menuliskannya sebagai dokumen.

Versi 1.1 · 8 September 2026 · Status: Draft untuk Review
Disusun oleh Faiza Kailani Kuswanto — Lead Developer

EMPAT STANDARD STACK YANG DISATUKAN

Frontend Web

React · Vite · Tailwind

Backend

NestJS · PostgreSQL · MongoDB

Mobile

Flutter · Riverpod · dio

Worker

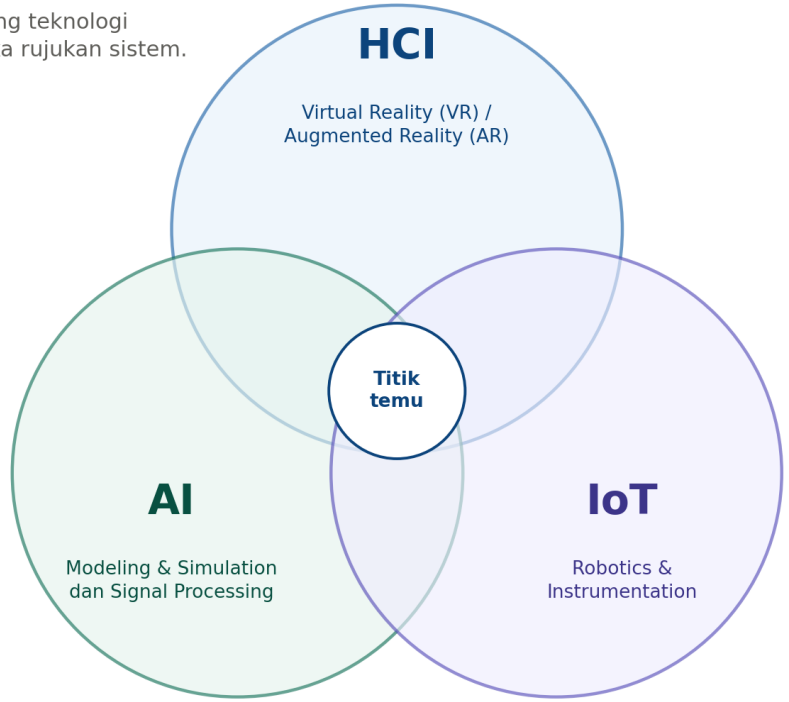
Go · Fiber · MQTT · RabbitMQ

Kerangka Rujukan: Konvergensi Tiga Bidang

Cara pandang yang dipakai Pak Ary. Big data lahir di titik temu tiga bidang teknologi.

BIG DATA

Konvergensi tiga bidang teknologi
yang menjadi kerangka rujukan sistem.



HCI

Virtual Reality dan Augmented Reality — sisi interaksi manusia dengan sistem.

AI

Modeling & simulation serta signal processing — sisi pengolahan dan penalaran.

IoT

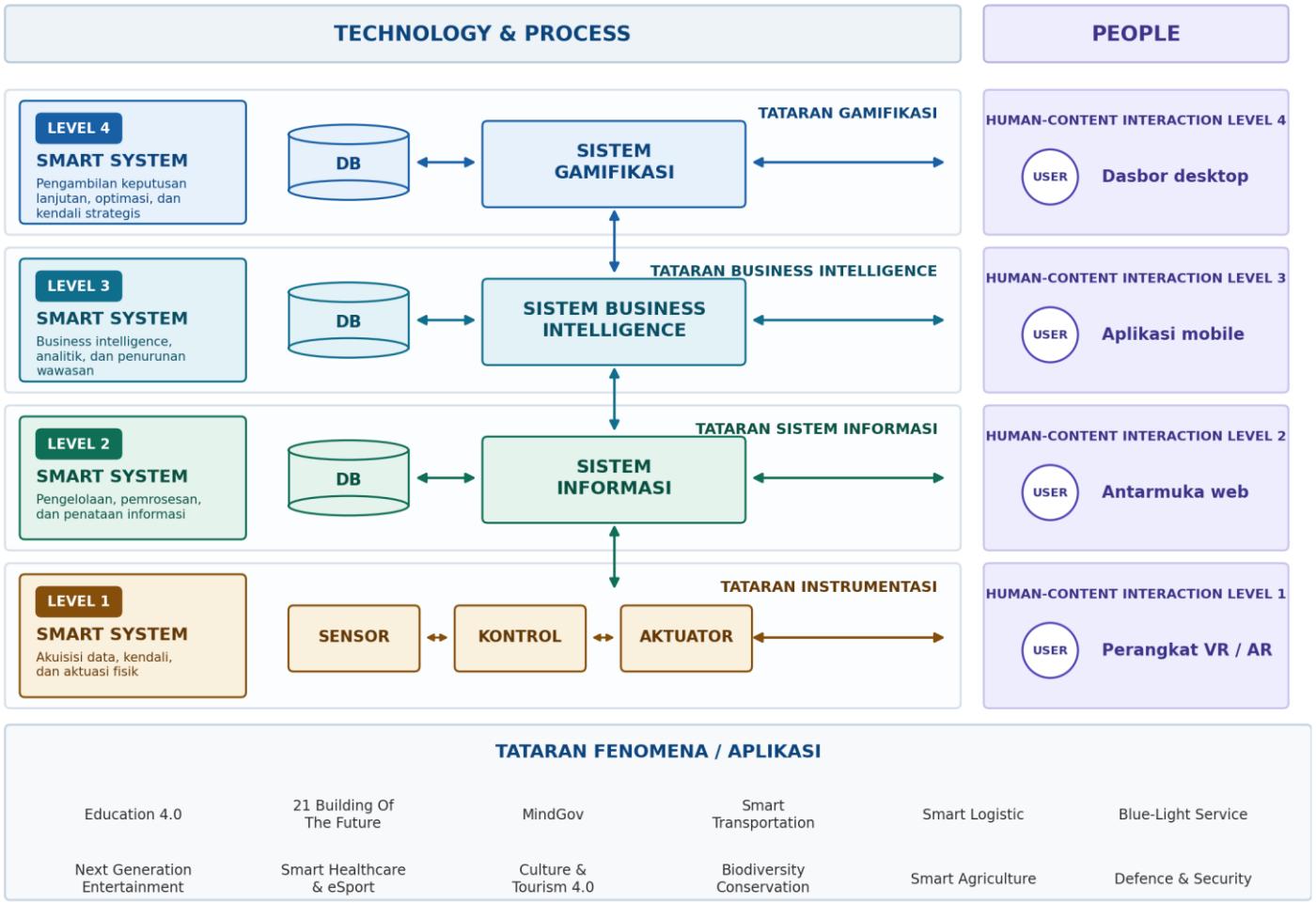
Robotics & instrumentation — sisi perangkat yang mengindera dan bertindak.

Mengapa ini penting

Hampir setiap proyek LSKK menyentuh sedikitnya dua dari tiga bidang ini. Kerangka ini menjelaskan mengapa arsitekturnya harus menampung ketiganya sekaligus.

Kerangka Rujukan: Empat Tataran Smart System

Menetapkan pada tataran mana sebuah sistem berada — dan sejauh mana ia perlu dibangun.



Level 4 — Gamifikasi

Keputusan lanjutan, optimasi, kendali strategis

Level 3 — Business Intelligence

Analitik dan penurunan wawasan

Level 2 — Sistem Informasi

Pengelolaan dan penataan informasi

Level 1 — Instrumentasi

Sensor, kontrol, aktuator

Cara memakainya

Di awal proyek, tentukan tataran targetnya. Tataran itu menentukan komponen mana pada arsitektur yang perlu dibangun, dan mana yang belum.

Isi Presentasi

Empat bagian, mengikuti struktur dokumen v1.0.

1 **Mengapa dokumen ini dibutuhkan** — Bagian 1

Apa yang sudah diatur empat Standard Stack, dan dua celah yang belum terjawab.

2 **Prinsip dan arsitektur sistem** — Bagian 2–3

Enam prinsip, peta layer, dan batas tanggung jawab antar layer.

3 **Pola pemrosesan dan visualisasi** — Bagian 4–6

Alur real-time, alur batch, dan dua mode dashboard.

4 **Penyusunan dokumen** — Bagian 7–11

Dokumen Engineering, Dokumen Teknis Rinci, dan tata kelolanya.

Mengapa Dokumen Ini Dibutuhkan

Empat Standard Stack sudah menjawab "teknologi apa yang dipakai". Dua pertanyaan berikutnya belum terjawab.

SUDAH DIATUR

Frontend Web v1.0

TypeScript · React · Vite · Tailwind · Ant Design · TanStack Query · Zustand

Backend v1.0

NestJS · PostgreSQL (TypeORM) · MongoDB (Mongoose) · Socket.IO · SSE · JWT

Mobile v1.0

Flutter · Riverpod · dio · freezed · go_router · secure storage · FCM

Worker v1.0

Go 1.25+ · Fiber v2 · MQTT (paho) · RabbitMQ (amqp091) · driver Mongo resmi

BELUM DIATUR

Bagaimana keempat layer menjadi satu sistem?

Bagaimana data mengalir dari sensor atau input pengguna sampai tampil sebagai visualisasi di layar — siapa menulis ke mana, dan lewat jalur apa.

Dokumen apa yang harus disusun, dengan kerangka seperti apa?

Kapan Dokumen Engineering dibuat, kapan Dokumen Teknis Rinci, siapa penyusunnya, dan apa bedanya untuk proyek baru versus proyek yang sudah berjalan.

Polanya bukan hal baru

Angkot to School, Rimba, dan Smart Farming sudah menjalankan pola ini. Dokumen v1.0 membakukan yang sudah terbukti agar tidak ditemukan ulang di tiap proyek.

Enam Prinsip Arsitektur

Berlaku untuk seluruh sistem LSKK. Rancangan yang melanggar salah satunya wajib disertai alasan tertulis pada Dokumen Engineering.

1 Pemisahan tanggung jawab

Setiap layer punya satu tanggung jawab utama dan tidak mengambil alih tugas layer lain.

2 Data perangkat lewat broker

Telemetri tidak pernah ditulis langsung ke basis data melalui endpoint REST.

3 Pemrosesan berat di worker

API mengembalikan data yang sudah siap saji, bukan menghitungnya saat request.

4 Satu pipeline, satu tanggung jawab

Worker dipecah per jenis pekerjaan; masukan, keluaran, dan persinggungannya ditulis eksplisit.

5 Satu kontrak data

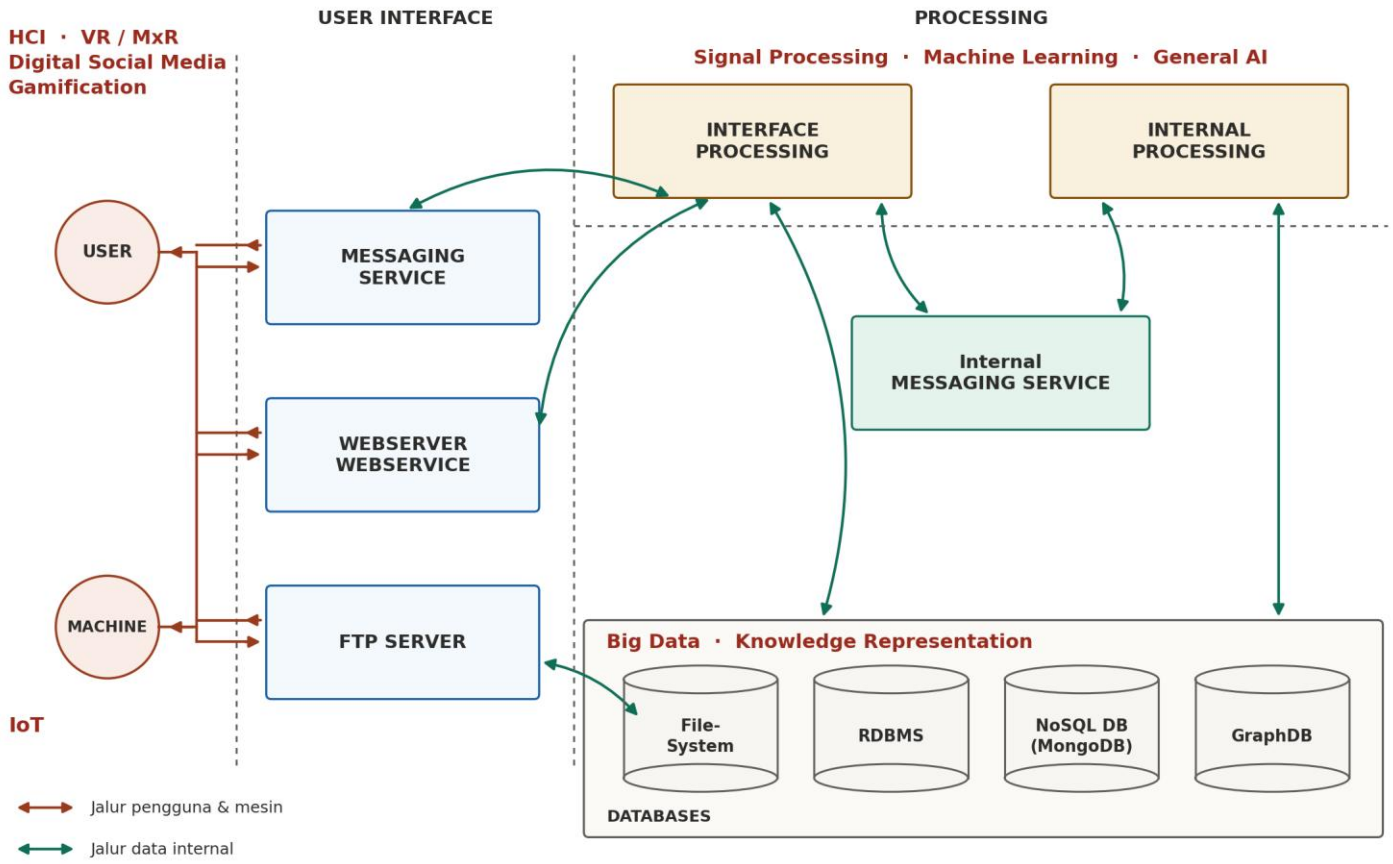
Response wrapper berbentuk sama di keempat layer; satu backend melayani semua klien.

6 Kondisi aktual dicatat apa adanya

Dokumentasi menggambarkan kode yang ada, bukan arsitektur yang diinginkan.

Arsitektur Referensi

Titik awal perancangan, bersumber dari slide arsitektur Pak Ary Setijadi Prihatmanto (PUSTEKHAN ITB).



TIGA ZONA

Pihak luar

Pengguna manusia dan mesin — tidak pernah mengakses pemrosesan atau basis data secara langsung.

Antarmuka

Satu-satunya pintu masuk dari luar: messaging service, webservice, dan file server.

Pemrosesan & penyimpanan

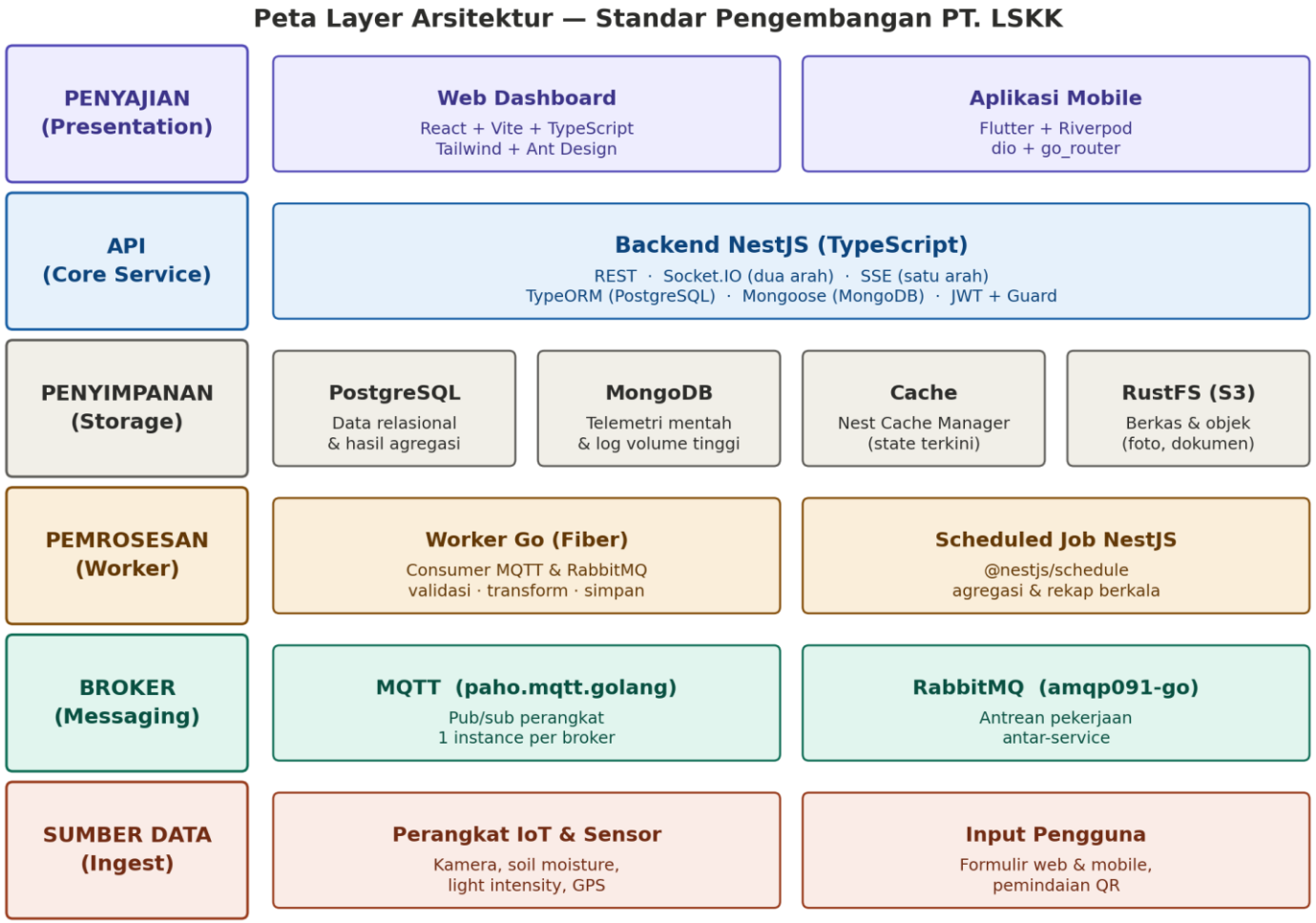
Interface processing, internal processing, messaging internal, dan seluruh basis data.

Anotasi merah = bidang kajian

HCI, VR/MxR, IoT, machine learning, big data — menandai di mana riset menempel, bukan komponen sistem.

Peta Layer Arsitektur

Realisasi konkret arsitektur referensi di LSKK, lengkap dengan teknologi wajib per layer.



CARA MEMBACA

- Kolom kiri adalah nama layer; kolom kanan adalah komponen di dalamnya.
- Urutan dari bawah ke atas mengikuti perjalanan data: masuk di Sumber Data, tampil di Penyajian.
- Warna menandai kelompok: hijau untuk broker, oranye untuk pemrosesan, biru untuk API dan penyajian.
- Seluruh teknologi di dalamnya bersifat wajib, bukan usulan yang bebas diganti tim.

Dua sumber, dua jalur

Perangkat IoT masuk lewat MQTT ke worker. Input pengguna masuk lewat REST ke API NestJS. Keduanya tidak ditukar.

Batas Tanggung Jawab Antar Layer

Yang boleh dan tidak boleh dilakukan setiap layer. Tabel ini dipakai sebagai acuan saat code review.

Layer	Bertanggung jawab atas	Tidak boleh melakukan
Penyajian	Menampilkan data, menangani interaksi, validasi masukan tingkat pertama	Menyimpan aturan bisnis; mengakses basis data langsung
API (NestJS)	Kontrak REST/WebSocket/SSE, autentikasi, otorisasi, aturan bisnis transaksional	Menjadi consumer telemetri perangkat bervolume tinggi
Pemrosesan	Konsumsi pesan, validasi payload, transformasi, agregasi berkala	Mengekspos endpoint bisnis untuk klien akhir
Penyimpanan	Menyimpan data sesuai bentuk dan karakter aksesnya	Memuat logika bisnis dalam trigger atau prosedur tersembunyi
Broker	Mengantarkan pesan antar komponen	Melakukan transformasi atau penyaringan data

Aturan wajib — akses broker terpusat

Seluruh consume dan publish, MQTT maupun RabbitMQ, melewati helper pkg/handler/message_broker. Handler consumer hanya unmarshal payload lalu memanggil service — tidak boleh ada logika bisnis di layer consumer.

Dua Jenis Sumber Data, Dua Jalur Masuk

Karakternya berbeda, sehingga perlakuannya berbeda. Menukar jalurnya menimbulkan masalah di kedua arah.

JALUR PERANGKAT

Perangkat IoT & Sensor

- Volume tinggi dan dikirim berkala
- Payload relatif seragam antar pengiriman
- Toleran terhadap keterlambatan beberapa detik
- Masuk melalui MQTT, dikonsumsi worker Go
- Contoh: kamera, soil moisture, light intensity, posisi GPS

JALUR PENGGUNA

Input Pengguna

- Volume rendah, terjadi karena aksi pengguna
- Membutuhkan konfirmasi seketika di layar
- Menuntut validasi ketat dan pesan error yang jelas
- Masuk melalui REST ke API NestJS
- Contoh: formulir pendaftaran, pemindaian QR, pencatatan pembayaran

Akibat bila jalurnya ditukar

Telemetri yang dipaksa lewat REST membebani API dengan pekerjaan yang bukan tugasnya. Input pengguna yang dipaksa lewat antrean kehilangan umpan balik seketika yang justru dibutuhkan pengguna.

Broker: MQTT dan RabbitMQ Berdampingan

Keduanya dipakai bersamaan dengan peran yang berbeda — bukan alternatif yang saling menggantikan.

PUB/SUB PERANGKAT

MQTT

- Pustaka: eclipse/paho.mqtt.golang
- Satu instance klien per broker (Local, Cloud, Camera, Soil Moisture, Light Intensity)
- Topik dipisah per jenis perangkat
- Setiap broker baru wajib punya prefix environment sendiri

ANTREAN ANTAR-SERVICE

RabbitMQ

- Pustaka: rabbitmq/amqp091-go
- Antrean pekerjaan antar-service, misalnya agregasi terjadwal
- Dipakai juga untuk perintah dari API ke worker
- Nama antrean dan exchange didaftarkan lewat helper terpusat

Tiga alasan data perangkat selalu melewati broker

Ketahanan

Lonjakan kiriman tertahan di antrean, bukan membanjiri basis data.

Kemandirian rilis

Worker dapat di-deploy ulang tanpa memutus jalur pengiriman data.

Kejelasan jejak

Satu titik masuk membuat penelusuran masalah jauh lebih mudah.

Pemrosesan: Worker Go atau Scheduled Job NestJS

Keduanya sah. Yang membedakan bukan selera, melainkan sifat pekerjaannya.

MICROSERVICE TERPISAH

Worker Go (Fiber)

- Konsumsi pesan berkelanjutan dari MQTT dan RabbitMQ
- Pekerjaan yang menuntut throughput tinggi
- Menyediakan health check dan simulator — bukan endpoint bisnis
- Tetap berjalan meskipun backend sedang di-deploy ulang

DI DALAM BACKEND

Scheduled Job NestJS

- Berjalan dengan @nestjs/schedule di aplikasi backend yang sama
- Pekerjaan berkala yang erat dengan aturan bisnis dan data relasional
- Contoh Angkot to School: flush posisi armada, deteksi keterlambatan
- Berbagi entitas dan service yang sudah ada di backend

Pedoman pemilihan

Berhubungan dengan perangkat, bervolume tinggi, atau perlu tetap hidup saat backend di-deploy ulang → worker Go. Erat dengan entitas relasional dan aturan bisnis yang sudah ada di backend → scheduled job NestJS.

Pekerjaan reaktif bukan pekerjaan terjadwal — deteksi yang harus segera diketahui berjalan setiap pesan diterima, dan ditulis pada bagian logika bisnis, bukan tabel scheduled jobs.

Penyimpanan: Dipilih Menurut Bentuk Data

PostgreSQL dan MongoDB keduanya resmi pada Backend Standard Stack. Dalam satu sistem, keduanya lazim dipakai bersamaan.

PostgreSQL

Data relasional yang integritasnya harus dijaga, serta tabel ringkasan hasil agregasi.

TypeORM · typeorm-transactional untuk operasi lintas tabel

MongoDB

Telemetri mentah, log perangkat, koleksi bervolume tinggi dengan struktur yang dapat berubah.

Mongoose di backend · driver resmi Go tanpa ODM di worker

Cache

State terkini yang sering dibaca namun tidak perlu ditulis ke basis data setiap saat.

Nest Cache Manager — bukan raw Redis client

RustFS (S3)

Berkas dan objek: foto bukti, lampiran, hasil ekspor laporan.

@aws-sdk/client-s3 dengan presigned URL

Pertanyaan penentu saat memulai proyek baru

Apakah data ini memiliki relasi antar entitas yang integritasnya harus dijaga? → PostgreSQL. Atau merupakan catatan bervolume tinggi yang dibaca sebagai deret waktu? → MongoDB.

Konsekuensi praktis: kueri agregasi untuk dashboard tidak dihitung ulang setiap halaman dibuka, melainkan disiapkan lebih dulu oleh worker sebagai tabel ringkasan.

Layanan AI: Celah yang Belum Tertutup

Keempat Standard Stack mengatur TypeScript, Go, dan Dart. Seluruh pekerjaan AI dan computer vision LSKK berjalan di Python.

KEADAAN SEKARANG

Dokumen AI menyatakan diri di luar standar

- Bird Detection: "aplikasi desktop berada di luar cakupan Development Standard Stack LSKK"
- ATCS: worker Python "didokumentasikan untuk kelengkapan arsitektur" saja

KEDUDUKANNYA SUDAH JELAS

Layanan AI = Internal Processing

- Pada arsitektur referensi Pak Ary, blok yang tidak pernah diakses langsung dari luar
- Konsekuensinya: layanan AI tidak pernah berada di jalur request-response API

Python adalah pengecualian yang disengaja, bukan penyimpangan

Prinsip 2.3 menempatkan pemrosesan berat di worker. Layanan AI adalah bentuk paling berat dari pemrosesan itu — penempatannya di luar backend justru mengikuti prinsip, bukan melanggarnya.

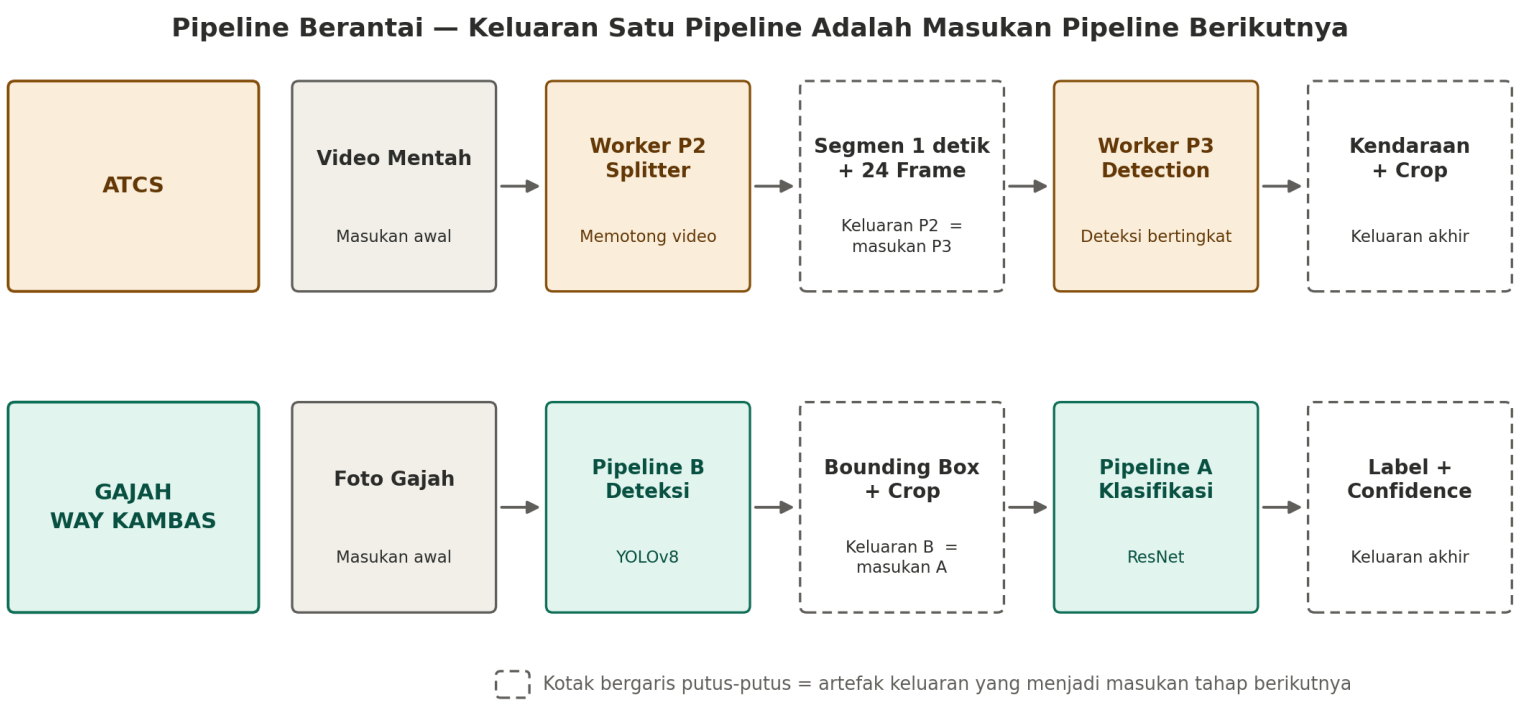
Yang dibakukan Bagian 5

Bukan boleh-tidaknya memakai Python, melainkan bagaimana layanannya dibangun dan disambungkan ke sistem: penempatan inference, tahapan pembangunan, pengelolaan artefak model, dan kontrak dengan sistem.

Prinsip Pipeline Berantai

Aturan yang disampaikan Pak Ary setiap kali pekerjaan AI ditugaskan — dan dasar seluruh Bagian 5.

"Proses harus dikerjakan per pipeline. Setiap pipeline menghasilkan keluaran, dan keluaran itulah yang dipakai sebagai masukan pipeline berikutnya."



Masukan & keluaran eksplisit

Bukan "memproses gambar", tapi bentuk artefak yang dihasilkan

Artefak yang dapat diperiksa

Berkas, dokumen basis data, atau pesan — bukan variabel dalam memori

Tidak saling memanggil langsung

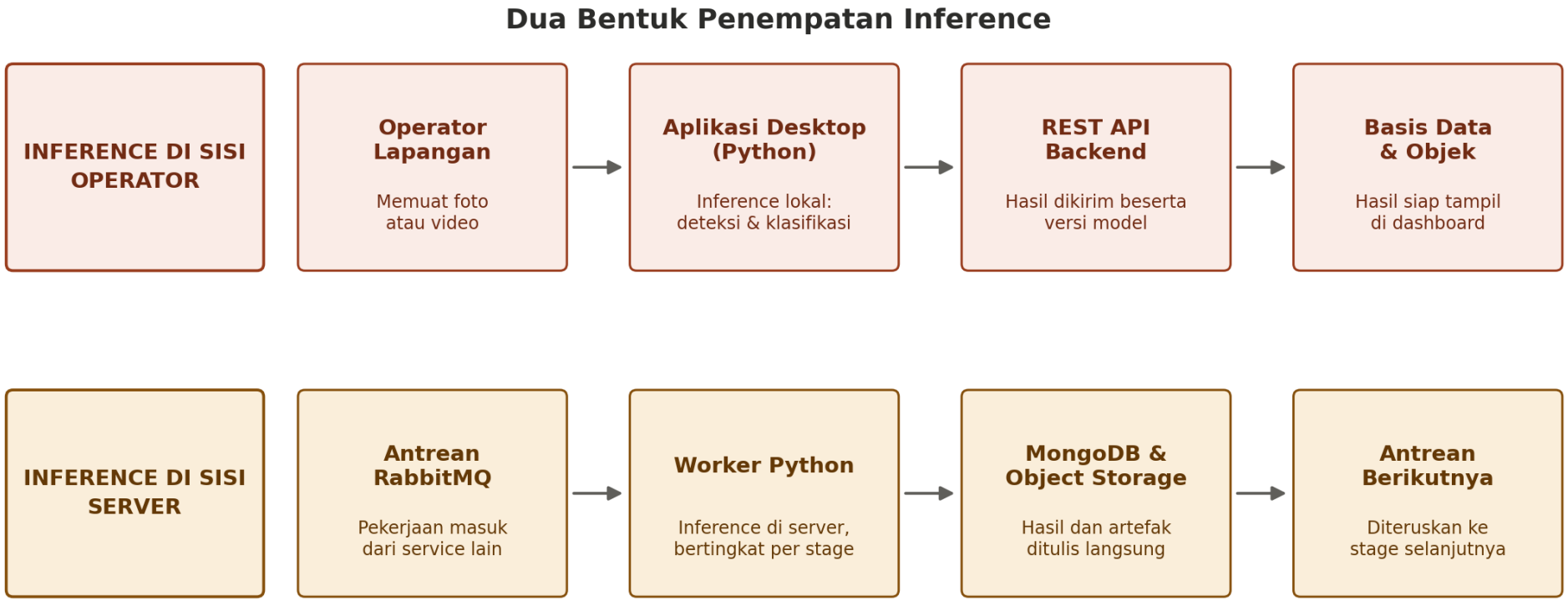
Disambung lewat artefak dan antrean, tiap tahap bisa diuji sendiri

Dapat dijalankan ulang

Tahap tiga gagal? Tahap satu dan dua tidak perlu diulang

Dua Bentuk Penempatan Inference

Keduanya sudah berjalan di proyek nyata. Yang menentukan bukan kecanggihan model, melainkan di mana datanya berada.



Keduanya sah. Yang menentukan bukan kecanggihan model, melainkan di mana datanya berada dan seberapa andal jaringannya.

Dipakai bila jaringan di lokasi tidak andal atau data mentah terlalu besar untuk diunggah

Gajah Way Kambas · Bird Detection

Dipakai bila data sudah di server, volume tinggi, dan kapasitas perlu diskalakan

ATCS worker P2 · P3

Ketentuan Wajib Layanan AI

Sebagian besar kegagalan yang ditemukan pada review bukan berasal dari modelnya, melainkan dari cara model dikelola.

ARTEFAK MODEL

Model menyatu dengan paket distribusi

Tidak diunduh saat runtime — pada ATCS ini jadi titik kegagalan startup

Varian model sama di semua tempat

Kode, Dockerfile, dan konfigurasi — ATCS sempat tidak sinkron

Versi model ikut di setiap hasil

Pola versiModel Gajah Way Kambas: hasil lama bisa ditelusuri ke modelnya

Dependensi dikunci pada versi tertentu

Pustaka CV dan ML sering berubah perilaku antar versi minor

KONTRAK DENGAN SISTEM

APLIKASI DESKTOP

Ia klien sistem, bukan sistem sendiri

- REST dengan IResponseEntity dan JWT seperti klien lain
- Kegagalan inference = pemberitahuan, bukan error fatal

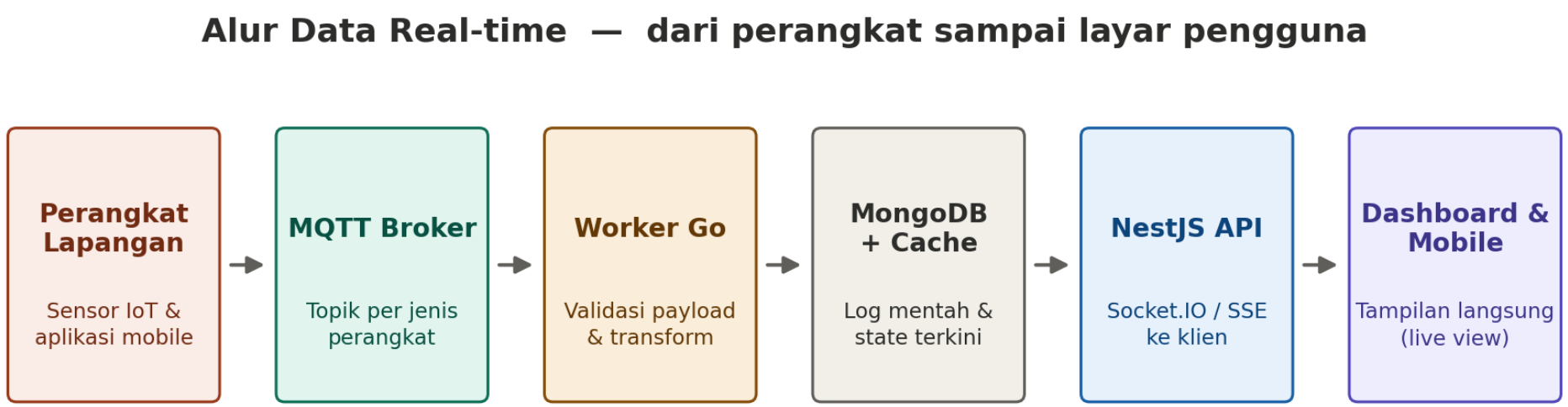
WORKER PYTHON

Siklus status yang terlihat

- Job: dibuat → berjalan → selesai (pola Bird Detection)
- Kegagalan dikembalikan ke antrean, bukan di-ack diam-diam

Alur Data Real-time

Dari perangkat sampai layar pengguna. Enam tahap, masing-masing dengan komponen penanggung jawab yang jelas.



Data perangkat tidak pernah ditulis langsung ke basis data oleh API; seluruh penulisan telemetri melewati broker dan worker.

Sediakan jalur cadangan

Endpoint REST untuk membaca state terakhir sebagai fallback bila WebSocket klien terputus.

Jangan tulis setiap pesan ke basis data

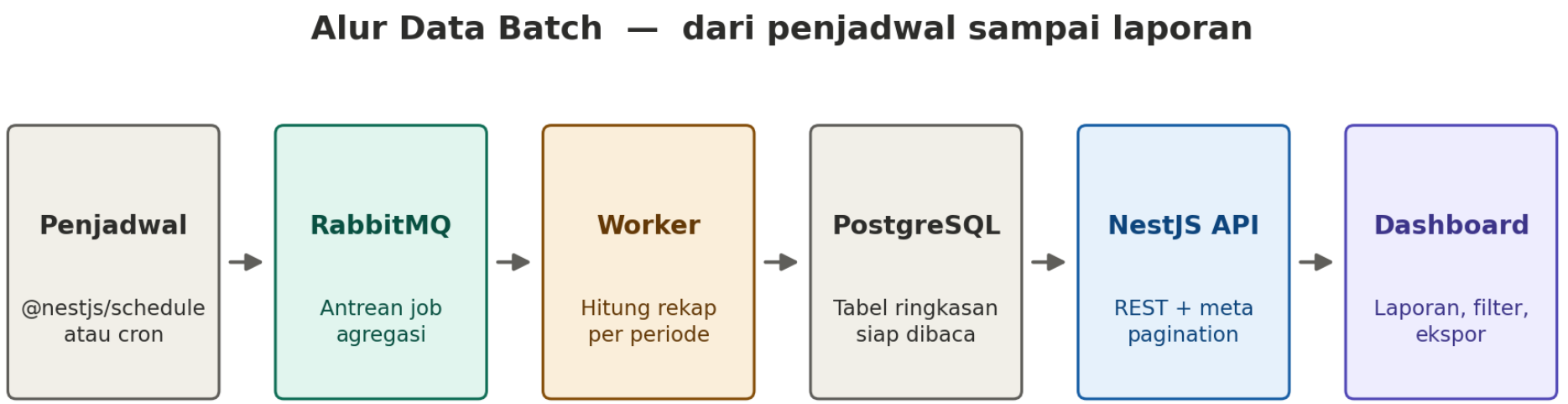
Tampung di cache, lalu flush berkala. Penulisan per beberapa detik membebani tanpa manfaat sebanding.

Deteksi perangkat tidak aktif

Pemeriksaan berkala menandai perangkat yang berhenti mengirim — pada Smart Farming setiap 60 detik.

Alur Data Batch

Memindahkan beban perhitungan dari saat dashboard dibuka ke saat penjadwal berjalan.



Kueri agregasi berat dijalankan di worker dan disimpan sebagai tabel ringkasan, bukan dihitung ulang setiap kali dashboard dibuka.

Flush data sementara

Memindahkan state terkini dari cache ke tabel log

Setiap 1 menit

Deteksi kondisi

Membandingkan waktu berjalan dengan jadwal untuk mendeteksi keterlambatan

Setiap 1–2 menit pada jam operasional

Agregasi pelaporan

Menghitung rekap harian, mingguan, atau bulanan ke tabel ringkasan

Harian di luar jam sibuk

Pembersihan

Menghapus token kedaluwarsa atau log yang melewati masa simpan

Harian, dini hari

Menentukan Pola yang Dipakai

Empat pertanyaan yang cukup untuk menentukan bentuk pipeline sebuah fitur baru.

Apakah pengguna perlu melihat perubahan dalam hitungan detik?	**YA**	Alur real-time	**TIDAK**	Alur batch
Apakah datanya berasal dari perangkat atau pengiriman berkala?	**YA**	Masuk melalui broker	**TIDAK**	Masuk melalui REST API
Apakah perhitungannya memakan waktu lebih dari satu detik?	**YA**	Pindahkan ke worker, simpan hasilnya	**TIDAK**	Boleh dihitung saat request
Apakah hasilnya sama untuk banyak pengguna?	**YA**	Hitung sekali, simpan sebagai ringkasan	**TIDAK**	Hitung sesuai konteks pengguna

Dua Mode Dashboard

Dipisahkan sejak perancangan, meskipun keduanya tampil dalam satu layar bagi pengguna.

Aspek	Mode Live	Mode Laporan
Tujuan	Memantau keadaan yang sedang berlangsung	Menganalisis data historis
Sumber data	Socket.IO / SSE, REST sebagai fallback	REST dengan pagination dan filter
Basis data	Cache dan MongoDB	PostgreSQL, umumnya tabel ringkasan
State	Zustand (web) / Riverpod (mobile)	TanStack Query (web) / provider Riverpod
Rentang waktu	Saat ini sampai beberapa jam terakhir	Harian, mingguan, bulanan, atau rentang bebas
Ekspor	Tidak tersedia	Tersedia (unduhan berkas)

Kesalahan yang sering terjadi

Menyatukan kedua mode dalam satu halaman dengan satu sumber data. Akibatnya salah satu selalu dikorbankan: mode live menjadi lambat karena menarik data historis, atau mode laporan tidak dapat difilter karena hanya menerima aliran realtime.

Satu Kontrak Data untuk Semua Layer

Bentuknya boleh berbeda menurut bahasa, strukturnya tidak boleh berbeda.

KONTRAK BERSAMA

```
interface IResponseEntity<T> {
  code: number;
  status: boolean;
  message: string;
  data?: T;
  meta?: ImetaPagination;
}
```

Backend (NestJS)	IResponseEntity<T> dibentuk ResponseInterceptor global. Controller cukup mengembalikan data atau { data, meta }.
Frontend (React)	Tipe response Axios memakai definisi yang sama, agar kontrak tidak pernah tidak sinkron dengan backend.
Mobile (Flutter)	ResponseEntity<T> dengan getter displayMessage — message dapat berupa String atau List<String>.
Worker (Go)	ResponseEntity[T] dan ResponseError[T]; wajib lewat helper SuccessResponse/ErrorResponse, bukan ctx.JSON().

BERLAKU LINTAS LAYER JUGA

Error handling

Satu global exception filter di backend, satu ErrorHandler terpusat di worker. Response gagal menyertakan path endpoint.

Autentikasi

JWT + Passport dengan rotasi refresh token. WebSocket diautentikasi lewat auth handshake, bukan query string.

Otorisasi

Guard berbasis peran. Setiap proyek wajib punya matriks akses per modul dan per peran.

Dua Jenis Dokumen

Satu menjawab "apa dan mengapa", satu menjawab "bagaimana persisnya".

SATU PER PROYEK

Dokumen Engineering

- Menjawab: sistem apa yang dibangun dan mengapa dirancang demikian
- Pembaca: seluruh tim, klien, dan pemangku kepentingan non-teknis
- Isi: latar belakang, lingkup MVP, arsitektur, modul per peran, timeline, risiko
- Disusun Lead Developer, sebelum baris kode pertama ditulis

SATU PER LAYER

Dokumen Teknis Rinci

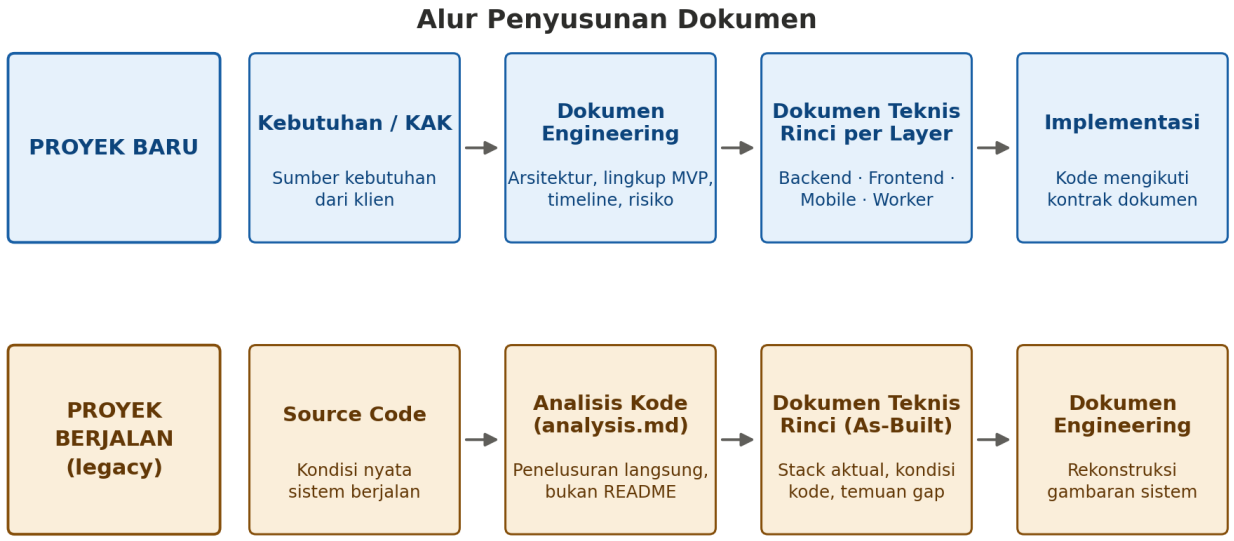
- Menjawab: bagaimana persisnya sistem itu dibangun
- Pembaca: developer pada layer yang bersangkutan
- Isi: skema basis data, kontrak API, matriks akses, logika bisnis, checklist rilis
- Disusun Lead Developer, ditinjau developer layer terkait

Fungsinya sebagai kontrak kerja

Frontend Developer dapat mulai integrasi berdasarkan kontrak API pada Dokumen Teknis Rinci Backend, tanpa menunggu backend selesai dibangun.

Dua Alur Penyusunan Dokumen

Perbedaan mendasarnya pada sumber kebenaran: rencana desain versus kondisi kode.



Kedua alur menghasilkan dokumen dengan kerangka yang sama; perbedaannya pada sumber kebenaran — rencana desain versus kondisi kode.

PROYEK BARU

Dokumen mendahului kode dan berfungsi sebagai kontrak. Perubahan besar pada Dokumen Engineering berimplikasi ke seluruh dokumen turunannya.

PROYEK BERJALAN

Kode mendahului dokumen. Dokumen berfungsi sebagai potret kondisi pada tanggal tertentu — bersifat deskriptif, tanpa rencana perbaikan.

Pencantuman tanggal analisis bersifat wajib

Tanpa pernyataan tanggal, dokumen as-built akan dianggap selalu berlaku dan menyesatkan pembaca beberapa bulan kemudian.

Kerangka Dokumen Engineering

Dua belas bagian, diturunkan dari Dokumen Engineering Angkot to School v1.3. Nomor bagian dipertahankan agar rujukan silang antar proyek konsisten.

1 Pendahuluan

2 Gambaran Umum Sistem

3 Arsitektur Sistem

4 Modul & Fitur per Peran

5 Ruang Lingkup MVP

6 Desain Basis Data

7 Rancangan API

8 Struktur Tim

9 Timeline

10 Kebutuhan Non-Fungsional

11 Risiko & Mitigasi

12 Lampiran

BAGIAN 5 — RUANG LINGKUP MVP ADALAH YANG PALING MENENTUKAN

P0

Wajib ada saat rilis

- Sistem tidak dapat dipakai sama sekali tanpa fitur ini — autentikasi, data master, fitur inti yang menjadi alasan sistem dibangun

P1

Menyusul, dengan alasan

- Menunggu keputusan pihak luar atau berisiko tinggi — alasan penundaan wajib ditulis, karena alasan itulah yang menentukan kapan bisa dikerjakan

Dokumen Teknis Rinci per Layer

Empat bagian pertama berbentuk sama di semua layer, agar developer yang berpindah dokumen tidak perlu menyesuaikan diri.

KERANGKA BERSAMA

Pendahuluan

Tujuan, ruang lingkup, dan cara menggunakan dokumen — termasuk bagian mana yang dibaca lebih dulu.

Tech stack & struktur

Merujuk Standard Stack terkait, dinyatakan sebagai ketentuan wajib.

Kontrak data

Response wrapper, interceptor, dan pola pengambilan data.

Checklist deploy

Daftar periksa spesifik layer tersebut sebelum rilis.

BAGIAN KHUSUS PER LAYER

Backend	Skema basis data lengkap · Modul & tanggung jawab · Kontrak API per modul · Matriks akses berbasis peran · Logika bisnis kunci · Event WebSocket · Pemicu notifikasi · Scheduled jobs · Seed & migrasi
Frontend	Strategi routing & route guard · Axios interceptor · Custom hook TanStack Query · Pembagian state · Struktur halaman per peran · Komponen reusable · Integrasi WebSocket & SSE · Design system
Mobile	Latar belakang pemisahan platform · go_router guard · dio interceptor · Repository & provider Riverpod · Halaman per peran beserta urutan prioritas · Tracking latar belakang · Push notification · Rilis
Worker	Struktur folder · Daftar consumer beserta topik & koleksi tujuan · Konvensi message broker · Konvensi service & DI · Konvensi config & environment · Endpoint health dan simulator · Graceful shutdown

Menulis Temuan pada Dokumen As-Built

Rumusan yang baik menyebutkan fakta, letaknya, dan implikasinya — tanpa menghakimi dan tanpa melompat ke solusi.

RUMUSAN YANG TEPAT	RUMUSAN YANG PERLU DIHINDARI
CORS origin: * pada worker — terlalu permisif untuk produksi.	Developer sebelumnya ceroboh menyetel CORS.
Rate limiter belum diimplementasikan; wajib ditambahkan sebelum endpoint diekspos publik (Worker Standard Stack Bagian 1).	Perlu segera dipasang rate limiter di sprint depan.
Clean Architecture yang digambarkan README belum diimplementasikan; struktur aktual masih layered. Perlu keputusan: migrasi atau perbarui README.	README salah.
Beberapa kredensial pada .env.example tampak seperti nilai produksi, bukan placeholder.	Kredensial produksi bocor di repository.

Mengapa temuan dan rencana perbaikan dipisahkan

Bila dicampur, setiap temuan langsung terbaca sebagai "sudah ada solusinya" — padahal solusi itu belum tentu dikerjakan. Dokumen as-built harus dapat dipercaya sebagai potret kondisi. Itulah nilainya.

Tata Kelola Dokumen

Siapa menyusun apa, siapa meninjaunya, dan kapan nomor versi dinaikkan.

Dokumen	Penyusun	Peninjau	Waktu penyusunan
Dokumen Engineering	Lead Developer	Atasan/pembimbing teknis, klien untuk bagian lingkup	Sebelum pengembangan dimulai
Teknis Rinci — proyek baru	Lead Developer	Developer layer bersangkutan	Setelah Dok. Engineering disetujui
Teknis Rinci — as-built	Lead Developer / developer ditugaskan	Developer yang memelihara sistem	Saat masuk cakupan pemeliharaan
Standard Stack per layer	Lead Developer	Seluruh developer layer bersangkutan	Berkala atau saat teknologi berubah

ATURAN PENOMORAN VERSI

Perbaikan redaksi

Tanpa mengubah ketentuan

Tidak perlu naik versi; perbarui tanggal

Penambahan ketentuan

Tidak membatalkan yang lama

Naik versi minor (v1.2 → v1.3)

Perubahan arsitektur

Membatalkan ketentuan sebelumnya

Naik versi mayor (v1.3 → v2.0)

Checklist Kepatuhan Standar

Diperiksa pada review desain proyek baru, sebelum implementasi dimulai. Checklist ini disalin ke tiap Dokumen Engineering.

- [ ] 1 Lingkup MVP sudah dilabeli P0/P1 beserta alasan penundaan
- [ ] 2 Tech stack sesuai Standard Stack; penyimpangan disertai alasan tertulis
- [ ] 3 Sumber data perangkat masuk lewat broker, bukan endpoint REST
- [ ] 4 Pemrosesan berat berada di worker atau scheduled job
- [ ] 5 Response wrapper seragam di seluruh layer, termasuk worker
- [ ] 6 Matriks akses berbasis peran sudah disusun untuk seluruh modul
- [ ] 7 Mode live dan mode laporan dipisahkan sejak perancangan
- [ ] 8 Seluruh endpoint list mendukung pagination sesuai kontrak
- [ ] 9 Dokumen Teknis Rinci tersedia untuk setiap layer yang dikerjakan
- [ ] 10 Checklist sebelum deploy sudah disusun dan disepakati tim

LANGKAH BERIKUTNYA

Empat hal yang perlu diputuskan bersama

Butir-butir berikut merupakan usulan pada versi 1.0 dan belum bisa ditetapkan sepihak.

1

Kadensi peninjauan Standard Stack

Ditinjau per kuartal, atau hanya saat ada perubahan teknologi?

2

Kewajiban dokumen untuk proyek kecil

Apakah proyek di bawah durasi tertentu cukup punya Dokumen Engineering saja?

3

Penyimpanan dan penomoran dokumen

Lokasi penyimpanan bersama serta konvensi penamaan berkas antar proyek.

4

Ambang migrasi Clean Architecture di worker

Menindaklanjuti known gap pada Worker Standard Stack Bagian 10.

Catatan untuk didiskusikan

Bagian 11.1 menempatkan Lead Developer sebagai penyusun hampir seluruh dokumen. Realistis untuk sekarang, namun dengan 15+ proyek aktif hal ini berpotensi menjadi hambatan. Perlu dipertimbangkan apakah Dokumen Teknis Rinci per layer dapat disusun developer layer masing-masing, dengan Lead Developer sebagai peninjau.